

🔗 See Topic 06 → Linked List Masterclass

🎯 GOAL: Master LIFO (Last In, First Out) thinking, recognize matching bracket/NGE patterns instantly, and master Monotonic Stacks in $O(N)$ time!

📦 REUSABLE STACK UTILITIES & APIS

ArrayDeque as Stack (Fast & Modern):

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(val); // Push onto top
int top = stack.pop(); // Pop top element
int peek = stack.peek(); // Peek top without popping
boolean empty = stack.isEmpty();
```

Monotonic Stack Template (Next Greater):

```
int[] nextGreater(int[] nums) {
    int[] res = new int[nums.length];
    Arrays.fill(res, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
            res[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return res;
}
```

💡 AHA MOMENT #1: LAST IN, FIRST OUT (LIFO) REVERSAL & PAIR MATCHING

A Stack is like a stack of plates: you can ONLY add to the top (**push**) or remove from the top (**pop**). This structure naturally remembers the **most recent state**, making it the perfect tool for matching nested pairs (e.g. `()`, `{}`), undo history, and finding Next Greater Elements!

1 CORE STACK OPERATIONS ($O(1)$ TIME)

- `push(x)`: Adds element 'x' to top of stack ($O(1)$).
- `pop()`: Removes and returns top element ($O(1)$).
- `peek()`: Views top element without removing ($O(1)$).
- `isEmpty()`: Returns true if stack contains 0 elements ($O(1)$).

2 JAVA MEMORY CALL STACK MODEL

Every method call in Java creates a **Stack Frame** on the JVM Call Stack storing local variables. When method returns, its frame is popped! Stack overflow happens when recursion exceeds JVM stack limit!

📊 VISUALIZING STACK PUSH / POP GROWTH

1. PUSH(10), PUSH(20), PUSH(30): 30 | ← TOP (peek/pop) | 20 | | 10 | ← BOTTOM 2. POP() → Returns 30: 20 | ← NEW TOP | 10 | |

🔑 KEY TAKEAWAY: Whenever a problem requires processing elements in reverse order of arrival or matching nested structures, reach for a Stack!

💡 **AHA MOMENT #2: WHY java.util.Stack IS LEGACY & DEPRECATED IN FAANG INTERVIEWS**

Never use `java.util.Stack` ! It was created in Java 1.0 extending `Vector` . Because it inherits vector indexing methods like `stack.get(index)` , it breaks LIFO encapsulation! Furthermore, every method is `synchronized` , adding unneeded locking overhead.

FAANG PREFERRED CHOICE: `Deque<Integer> stack = new ArrayDeque<>();` !

1 FAANG PREFERRED: ARRAYDEQUE METHOD MAPPING

Stack Operation	ArrayDeque Method	Time Complexity
Push Element	<code>stack.push(x)</code>	$O(1)$ amortized
Pop Element	<code>stack.pop()</code>	$O(1)$
Peek Top	<code>stack.peek()</code>	$O(1)$
Check Empty	<code>stack.isEmpty()</code>	$O(1)$

2 STACK PERFORMANCE BENCHMARK

Class	Synchronization	Cache Locality	FAANG Verdict
<code>ArrayDeque</code>	Non-synchronized	Contiguous Array (High)	✅ BEST / PREFERRED
<code>LinkedList</code>	Non-synchronized	Heap Nodes (Low)	⚠️ Extra node overhead
<code>java.util.Stack</code>	Synchronized	Contiguous Vector	❌ LEGACY / AVOID

 JAVA ARRAYDEQUE STACK SNIPPET

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
int topVal = stack.peek(); // 20
int popped = stack.pop();  // 20
boolean empty = stack.isEmpty(); // false
```

⚠️ **COMMON BUG:** Calling `stack.pop()` or `stack.peek()` on an empty `ArrayDeque` throws a `NoSuchElementException` ! Always check `!stack.isEmpty()` first!

1 BALANCED PARENTHESES MECHANICS

- **Opening Brackets** (, {, [: Push onto stack.
- **Closing Brackets**), },] : Peek top of stack. If stack is empty OR top bracket does not match closing type → Invalid! Pop top on valid match.
- **Final Check**: If stack is empty at end → Valid!

2 REVERSE POLISH NOTATION (RPN)

- In RPN (Postfix notation e.g. ["2", "1", "+", "3", "*"]):
- **Operands (Numbers)**: Push to stack.
 - **Operators** (+, -, *, /): Pop `b` (2nd operand), pop `a` (1st operand), evaluate `a op b`, and push result back to stack!

RPN EXPRESSION EVALUATION TRACE: ["2", "1", "+", "3", "*"]

1. Push 2, Push 1 → Stack: [2, 1] (Top: 1) 2. Operator '+' → Pop 1, Pop 2. Compute 2 + 1 = 3. Push 3 → Stack: [3] 3. Push 3 → Stack: [3, 3] (Top: 3) 4. Operator '*' → Pop 3, Pop 3. Compute 3 * 3 = 9. Push 9 → Stack: [9] Final Result = 9!

🚩 **KEY TAKEAWAY:** For operator evaluation in RPN, the **FIRST** popped element is the **SECOND** operand (`b = stack.pop()`, `a = stack.pop()`)!

💡 AHA MOMENT #3: MONOTONIC STACK (\$O(N)\$ NEXT GREATER / SMALLER ELEMENT)

A Monotonic Stack maintains elements in strictly sorted order (Monotonic Increasing or Monotonic Decreasing).

- **Monotonic Decreasing Stack:** Used to find **Next Greater Element (NGE)**! When current element `num > stack.peek()` , pop smaller elements because `num` is their Next Greater Element!
- Each element is pushed and popped AT MOST ONCE, guaranteeing \$O(N)\$ linear total runtime!

📋 STEP-BY-STEP NGE TRACE TABLE: INPUT = [2, 1, 5, 6, 2, 3]

Index i	Value nums[i]	Monotonic Decreasing Stack Action	NGE Result State
0	2	Push 2 → Stack: [2]	[-1, -1, -1, -1, -1, -1]
1	1	1 < 2 → Push 1 → Stack: [2, 1]	[-1, -1, -1, -1, -1, -1]
2	5	5 > 1 → Pop 1 (NGE of 1 is 5!). 5 > 2 → Pop 2 (NGE of 2 is 5!). Push 5 → Stack: [5]	[5, 5, -1, -1, -1, -1]
3	6	6 > 5 → Pop 5 (NGE of 5 is 6!). Push 6 → Stack: [6]	[5, 5, 6, -1, -1, -1]
4	2	2 < 6 → Push 2 → Stack: [6, 2]	[5, 5, 6, -1, -1, -1]
5	3	3 > 2 → Pop 2 (NGE of 2 is 3!). Push 3 → Stack: [6, 3]	[5, 5, 6, -1, 3, -1]

🚀 MONOTONIC RULE: To find Next Greater Element → Maintain Decreasing Stack. To find Next Smaller Element → Maintain Increasing Stack!

 STACK INTERVIEW DECISION TREE (MERMAID)

PRINTABLE STACK & MONOTONIC CHEAT SHEET			
Pattern Type	Primary Target Question	Stack Variant	Key Code Mechanism
Parentheses Matching	Valid Parentheses (LC 20)	Character Stack	Push open bracket, pop & match on closing bracket
Next Greater Element	Daily Temperatures (LC 739)	Monotonic Decreasing Stack	<code>while (!s.isEmpty() && arr[i] > arr[s.peek()]) pop()</code>
Next Smaller Element	Histogram Rectangle (LC 84)	Monotonic Increasing Stack	<code>while (!s.isEmpty() && arr[i] < arr[s.peek()]) pop()</code>
Expression Evaluation	Evaluate RPN (LC 150)	Operand Stack	Push numbers; on operator, pop <code>b</code> then <code>a</code> , push <code>a op b</code>
$O(1)$ \$ Min Tracking	Min Stack (LC 155)	Two Stacks or Diff Encoding	Auxiliary stack tracks minimum value at every depth

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

1. LEETCODE 20 — VALID PARENTHESES

 **Easy ★★★★★** Character Matching [Amazon](#) [Meta](#) [Google](#)

★ **FLOWCHART:** Open bracket → push corresponding close bracket. Close bracket → check `stack.peek()`!

Problem:

Given a string

`s`

containing

`()[]{}`

, determine if valid.

Example:

`s = "()[]{}"`

→ Output:

`true`

.

`s = "([)"`

→ Output:

`false`

.

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

2. LEETCODE 682 — BASEBALL GAME

 **Easy ★★★★★** Score Stack [Amazon](#)

Problem:

Track baseball scores using operations

`+`

`D`

`C`

, or integer scores.

```
public int calPoints(String[] operations) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String op : operations) {
        if (op.equals("+")) {
            int top = stack.pop(), newTop = top + stack.peek();
            stack.push(top); stack.push(newTop);
        } else if (op.equals("D")) stack.push(2 * stack.peek());
        else if (op.equals("C")) stack.pop();
        else stack.push(Integer.parseInt(op));
    }
    int sum = 0; for (int score : stack) sum += score;
    return sum;
}
```

3. LEETCODE 844 — BACKSPACE STRING COMPARE

 **Easy ★★★★★** String Editing [Google](#)

Problem:

Compare two strings

`s`

and

`t`

where

`#`

represents backspacing.

```
public boolean backspaceCompare(String s, String t) {
    return build(s).equals(build(t));
}
private String build(String str) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : str.toCharArray()) {
        if (c != '#') stack.push(c);
        else if (!stack.isEmpty()) stack.pop();
    }
    return String.valueOf(stack);
}
```

💡 **ELEGANT TRICK (LC 20):** Pushing expected CLOSING brackets onto stack makes closing matching a 1-line check `stack.pop() != c`!

4. LEETCODE 150 — EVALUATE REVERSE POLISH NOTATION

Medium ★★★★★
Operand Stack
Amazon
Meta
Google

★ **FLOWCHART:** Number → push. Operator → b=pop(), a=pop(), compute a op b, push result!

Problem:

Evaluate value of arithmetic expression in Reverse Polish Notation.

Example:

tokens = ["2", "1", "+", "3", "*"]

→ Output:

9

((2 + 1) * 3).

```
public int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.equals("+")) stack.push(stack.pop() + stack.pop());
        else if (t.equals("*")) stack.push(stack.pop() * stack.pop());
        else if (t.equals("-")) {
            int b = stack.pop(), a = stack.pop(); stack.push(a - b);
        } else if (t.equals("/")) {
            int b = stack.pop(), a = stack.pop(); stack.push(a / b);
        } else stack.push(Integer.parseInt(t));
    }
    return stack.pop();
}
```

5. LEETCODE 739 — DAILY TEMPERATURES

Medium ★★★★★
Monotonic Decreasing Stack
Amazon
Meta

★ **FLOWCHART:** Maintain Monotonic Decreasing Stack of indices! When T[i] > T[stack.peek()], pop and record i - prevIndex!

Problem:

Given array of temperatures, return array

answer

where

answer[i]

is number of days to wait for a warmer temperature.

Example:

temperatures = [73, 74, 75, 71, 69, 72, 76, 73]

→

[1, 1, 4, 2, 1, 1, 0, 0]

.

```
public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] res = new int[n];
    Deque<Integer> stack = new ArrayDeque<>(); // Stores indices
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] >
            temperatures[stack.peek()]) {
            int prevIdx = stack.pop();
            res[prevIdx] = i - prevIdx; // Calculate day distance!
        }
        stack.push(i);
    }
    return res;
}
```

💡 **AHA MOMENT (LC 739):** Storing INDICES instead of values in the Monotonic Stack allows calculating both the Next Greater Element AND the distance index difference 'i - prevIdx'!



6. LEETCODE 155 — MIN STACK

Medium ★★★★★ Auxiliary Min Stack Amazon Microsoft

Problem:

Design a stack supporting

push

pop

top

, and retrieving minimum element in $O(1)$ time.

```
class MinStack {
    private Deque<Integer> stack = new ArrayDeque<>();
    private Deque<Integer> minStack = new ArrayDeque<>();
    public void push(int val) {
        stack.push(val);
        val = Math.min(val, minStack.isEmpty() ? val : minStack.peek());
        minStack.push(val);
    }
    public void pop() { stack.pop(); minStack.pop(); }
    public int top() { return stack.peek(); }
    public int getMin() { return minStack.peek(); }
}
```

7. LEETCODE 853 — CAR FLEET

Medium ★★★★★ Monotonic Arrival Time Google

Problem: n cars heading to target. Return number of car fleets formed.**Formula:** $\text{time} = (\text{target} - \text{position}) / \text{speed}$. Sort cars by position descending!

```
public int carFleet(int target, int[] position, int[] speed) {
    int n = position.length;
    double[][] cars = new double[n][2];
    for (int i = 0; i < n; i++) {
        cars[i][0] = position[i];
        cars[i][1] = (double)(target - position[i]) / speed[i];
    }
    Arrays.sort(cars, (a, b) -> Double.compare(b[0], a[0])); // Position desc
    Deque<Double> stack = new ArrayDeque<>();
    for (double[] car : cars) {
        double time = car[1];
        if (stack.isEmpty() || time > stack.peek()) stack.push(time);
    }
    return stack.size(); // Number of fleets!
}
```

**AHA MOMENT (LC 853):** Sorting cars by position descending lets us evaluate arrival times from closest to furthest! If a car takes $\leq$ time of car ahead, it merges into that fleet!

8. LEETCODE 22 — GENERATE PARENTHESES


Medium
★★★★★
 Backtracking Stack
 [Amazon](#)
[Meta](#)
[Google](#)

Problem:

Given `n` pairs of parentheses, generate all combinations of well-formed parentheses.

```
public List<String> generateParenthesis(int n) {
    List<String> res = new ArrayList<>();
    backtrack(res, new StringBuilder(), 0, 0, n);
    return res;
}

private void backtrack(List<String> res, StringBuilder sb, int open, int close, int max) {
    if (sb.length() == max * 2) { res.add(sb.toString()); return; }
    if (open < max) {
        sb.append('('); backtrack(res, sb, open + 1, close, max);
        sb.deleteCharAt(sb.length() - 1);
    }
    if (close < open) {
        sb.append(')'); backtrack(res, sb, open, close + 1, max);
        sb.deleteCharAt(sb.length() - 1);
    }
}
```


Medium
★★★★★
 Collision Stack
 [Amazon](#)
[Google](#)

Problem:

Asteroids moving right (+val) or left (-val). Find state of asteroids after all collisions.

```
public int[] asteroidCollision(int[] asteroids) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (int ast : asteroids) {
        boolean destroyed = false;
        while (!stack.isEmpty() && ast < 0 && stack.peek() > 0) {
            if (stack.peek() < -ast) stack.pop(); // Pop smaller right-moving
            else if (stack.peek() == -ast) { stack.pop(); destroyed = true; break; }
            else { destroyed = true; break; }
        }
        if (!destroyed) stack.push(ast);
    }
    int[] res = new int[stack.size()];
    for (int i = res.length - 1; i ≥ 0; i--) res[i] = stack.pop();
    return res;
}
```


MEMORY BOX: Collision happens **ONLY** when stack top moves **RIGHT** ('> 0') and current asteroid moves **LEFT** ('< 0')!



10. LEETCODE 84 — LARGEST RECTANGLE IN HISTOGRAM

Hard ★★★★★ Monotonic Increasing Stack [Amazon](#) [Meta](#) [Google](#)**Problem:**

Given array of heights, return area of largest rectangle in histogram in $O(N)$ time.

```
public int largestRectangleArea(int[] heights) {
    int n = heights.length, maxArea = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i ≤ n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : (i - stack.peek() - 1);
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

11. LEETCODE 42 — TRAPPING RAIN WATER (STACK APPROACH)

Hard ★★★★★ Monotonic Decreasing Stack [Amazon](#) [Meta](#)**Problem:**

Compute trapped rain water after raining using Monotonic Decreasing Stack.

```
public int trap(int[] height) {
    int water = 0, current = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    while (current < height.length) {
        while (!stack.isEmpty() && height[current] >
            height[stack.peek()]) {
            int top = stack.pop();
            if (stack.isEmpty()) break;
            int distance = current - stack.peek() - 1;
            int boundedHeight = Math.min(height[current],
                height[stack.peek()]) - height[top];
            water += distance * boundedHeight;
        }
        stack.push(current++);
    }
    return water;
}
```

FAANG COACH TIP: For Histogram Rectangle, appending a dummy height of 0 at index 'N' forces the stack to flush and evaluate all remaining bars!

★ THE 4 GOLDEN LAWS OF STACKS

1. **Use ArrayDeque:** Always declare `Deque<Integer> stack = new ArrayDeque<>();` (Never use legacy `java.util.Stack`!).
2. **Monotonic Decreasing NGE:** Maintain decreasing stack to find Next Greater Element in $O(N)$ time.
3. **Store Indices in Stack:** Store array indices instead of raw values to compute distances (e.g. `i - stack.peek()`).
4. **Empty Stack Safety:** Always check `!stack.isEmpty()` before calling `pop()` or `peek()`!

🚫 WHEN STACK FAILS (DO NOT USE!)

- **Random Index Access:** Stack ONLY allows viewing top element ($O(1)$) → Use **Array / ArrayList**!
- **FIFO Processing (First Come First Served):** Stack reverses arrival order → Use **Queue / LinkedList**!

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

 Easy LC 20 Valid Parentheses	 Easy LC 682 Baseball Game	 Easy LC 844 Backspace Compare
 Med LC 150 Evaluate RPN	 Med LC 739 Daily Temperatures	 Med LC 155 Min Stack
 Med LC 853 Car Fleet	 Med LC 22 Generate Parentheses	 Med LC 735 Asteroid Collision
 Hard LC 84 Histogram Rectangle	 Hard LC 42 Trapping Rain Water	

👉 **Next Master Topic: Topic 08** → **Queue & Monotonic Queue Masterclass**

🎉 **TOPIC 07 STACK MASTERCLASS COMPLETE!** Turn the page to **Topic 08: Queue & Monotonic Queue Masterclass!**